

Actividad de Laboratorio: Interoperabilidad y Carga Masiva de Datos

Duración estimada: 35 minutos

Modalidad: Individual

Entregable: Un archivo en formato Markdown (Integracion_Datos.md) que contenga las respuestas de la evaluación conceptual y los bloques de código solicitados para la implementación práctica.

Parte 1: Evaluación Conceptual y Buenas Prácticas (10 minutos)

Con base en el material oficial de la **Sesión 20: Integración de Datos**, responde de manera concreta a las siguientes preguntas:

Formatos de Intercambio: Completa la siguiente tabla comparativa de formatos masivos según las ventajas y desventajas expuestas en la sesión:

Formato	Ventajas	Desventajas
CSV	_____	_____
XML	_____	_____

1. **Diferenciación de Procesos:** Explica la diferencia técnica entre los procesos de **Serialización** y **Deserialización** utilizando la librería nativa System.Text.Json.
2. **El Antipatrón del Rendimiento:** Explica brevemente en qué consiste el error común de rendimiento denominado "N+1" durante la lectura de un archivo masivo y qué estrategia de optimización por lotes (*Batching*) se debe aplicar para solucionarlo.

Parte 2: Implementación Práctica en C#

Imagina que estás desarrollando el módulo de conectividad para el *Sistema de Control Académico de la USAC*. Debes escribir dos fragmentos de código limpios en tu archivo de entrega:

Desafío 1: Consumo de Endpoints y Deserialización

Escribe un método asíncrono en C# que utilice la clase HttpClient para realizar una petición segura de tipo GET a la URL https://api.usac.edu/v1/alumnos. El código debe cumplir con las siguientes directrices de la sesión:

- Implementar el manejo de excepciones con un bloque try-catch y validar el código de estado usando EnsureSuccessStatusCode().
- Deserializar el payload JSON de la respuesta a un objeto de tipo Alumno utilizando JsonSerializerOptions con la propiedad de insensibilidad a mayúsculas y minúsculas

habilitada (PropertyNameCaseInsensitive = true).

Desafío 2: Endpoint para Carga Masiva CSV

Desarrolla un endpoint en C# ([HttpPost]) para un controlador web que reciba un archivo masivo mediante IFormFile. La lógica interna debe cumplir estrictamente con los siguientes estándares de rendimiento:

- Utilizar la clase StreamReader para procesar el archivo línea por línea de forma asíncrona (ReadLineAsync()), evitando el riesgo de saturación de la memoria RAM del servidor.
- Almacenar los registros mapeados en una lista intermedio y realizar la inserción completa en la base de datos en un solo bloque por lotes mediante AddRange() y una única llamada a SaveChangesAsync() al finalizar el ciclo de lectura.

Parte 3: Referencias Bibliográficas (Requisito Formal)

Para dar por válida la actividad, incluye al final de tu documento el registro formal de la fuente utilizada:

- Facultad de Ingeniería, USAC. (2026). *Sesión 20: Integración de Datos. Consumo de APIs Externas y Carga Masiva (CSV/XML)*. Laboratorio del curso Introducción a la Programación y Computación 2. Guatemala.